# Using the AIX Lab-Atlas skills

in Claude Code

| | |
|---|---|
| **For** | Designers who already have Claude Code open. Setup takes about 10 minutes, once per computer. |
| **Source** | https://github.com/RPalis/AIXLab-Atlas (branch `update-readme-retire-competitors-duplicate`). This is a team guide, not an official Anthropic document. |
| **Status** | Installer checked on macOS. Windows has not been tested yet. |

## What you are setting up

The repository holds 30 skills. Each skill is a folder with a `SKILL.md` file of instructions, for example `audit`, `brief` and `persona`. An installer copies the folders to one place on your computer and links them into Claude Code, so you can run any skill by name from any project. You need a terminal for the install, once.

## Step 1. Get the files

1. Open https://github.com/RPalis/AIXLab-Atlas. The work is on the branch `update-readme-retire-competitors-duplicate`, not `main`. Click the branch button and choose it.
2. Click the green **Code** button, then **Download ZIP**, and unzip it. Private repository: sign in with the GitHub account that was invited first. Direct link while signed in: `https://github.com/RPalis/AIXLab-Atlas/archive/refs/heads/update-readme-retire-competitors-duplicate.zip`.

> **CHECK** The unzipped folder contains `skills`, `README.md` and `install.sh`.

## Step 2. Install the skills

1. Open **Terminal** (Cmd+Space, type Terminal). Type `cd ` plus a space, drag the unzipped folder onto the window, press Enter.
2. Run the installer:

```
./install.sh --claude --no-open
```

1. If you see "permission denied", run `chmod +x install.sh` and try again.

> **CHECK** The last lines say "Successfully installed 30 skills!". The installer copied the skills to `~/.agents/skills` and linked each into `~/.claude/skills`. Nothing else changed. It needs a Linux-style shell, so on Windows use WSL, or copy the 30 folders from `skills` into `C:\Users\<you>\.claude\skills`; neither has been tested yet.

## Step 3. Check that Claude Code sees them

1. Open Claude Code in your project folder (desktop app: **Code** tab, then choose the folder; terminal: `cd` into it, then `claude`). Quit and reopen Claude Code first if it was already running during the install.
2. Type `/` and look at the list.

> **CHECK** The list includes skills such as `setup`, `brief`, `persona` and `audit`. If it shows none, quit Claude fully, reopen it, and check the installer said 30.

> **NOTE** `/help` is also a built-in Claude Code command, and the built-in wins. To run this suite's help, type: Use the AIX Lab-Atlas help skill. Every other skill name works as a slash command.

## Step 4. Invoke a skill

Type a slash, the skill name, and an optional mode or idea, then send. If you do not know where to start, ask for the help skill. It reads your project's files and recommends the next one to three skills, each tied to a file it found or did not find.

```
/brief an app to share weekly meal plans with housemates
/audit craft
/map flow
Use the AIX Lab-Atlas help skill
```

The slash is optional. "Run the audit skill in craft mode" does the same thing. Skills that need a mode ask for it if you do not give one.

## The commands

| Phase | Commands (modes in brackets) |
|---|---|
| Router | help [default, atlas]. See the note in step 3 about `/help`. |
| Before Discover | `/setup`, `/brief` |
| Discover | `/workshop` [plan, ingest], `/competitor-analysis` [discover, add, analyse], `/ux-benchmark`, `/interview`, `/interview-analysis`, `/persona` [card, day], `/journey` [map, blueprint, pain-points, solutions, experience], `/problem-statement`, `/metrics` |
| Decide | `/brainstorming`, `/information-architecture`, `/map` [objects, site, flow, audit, document], `/interrogate`, `/decisions` |
| Design | `/prototype`, `/styleguide` [default, forge], `/audit` [craft, heuristic, code, spec, a11y], `/refine`, `/storybook`, `/figma` [capture, import, sync] |
| Deliver | `/usability-test-plan`, `/package` [full, lite], `/showcase`, `/guide`, `/requirements` |
| Any time | `/wiki` [compile, note], `/skill-manager` |

## What each skill needs

Every skill states what it needs in its own `## Environment` section. In short:

| Needs | Skills |
|---|---|
| The conversation only (saving a file is optional) | brainstorming, interrogate, problem-statement, usability-test-plan, information-architecture, interview, interview-analysis, persona, journey, metrics, decisions, help |
| A file system | setup, brief, wiki, requirements, guide, styleguide, refine, figma (import, sync) |
| A shell and Node.js | prototype, storybook, package, showcase, skill-manager |
| Web search or browser tooling | competitor-analysis, ux-benchmark, showcase (screenshots) |
| A connected MCP server | figma (import, sync), workshop (FigJam board), map (FigJam export), requirements (tracker export) |

When something is missing, a skill says so, returns its output in the conversation, and never claims a file was written when it was not.

## Audit

Audit finds problems and ranks them. It never edits your design. Name a mode, or it asks.

- **craft:** five dimensions scored 0 to 4 (total out of 20): visual hierarchy and cognitive load, information architecture, consistency, responsive behavior, delight. It also walks 2 to 3 personas through the main task. The output is a task list ranked P0 to P3.
- **heuristic:** a deeper pass against 12 usability heuristics, each issue rated 0 to 4 and tied to a screen. Use it when the audit has to stand as evidence.
- **a11y:** WCAG 2.1 and 2.2 AA, criterion by criterion.
- **code:** token use, structure, reuse and semantic HTML.
- **spec:** the build against the approved design, screen by screen, at every breakpoint.

Craft and heuristic scores are different instruments, so never average or merge them. Run modes one at a time. To fix findings, run `/refine`, then audit again to see whether the score moved. For a ship or handoff decision, ask for an independent second pass and reconcile both lists.

## Plan

1. **Place the project.** Ask for the help skill. It reads which docs exist and recommends the next skill. An empty project gets `/setup`, then `/brief`.
2. **Follow the phases in order:** Discover, Decide, Design, Deliver. A project can skip ahead and backfill, for example by starting with a prototype and writing the research later.
3. **Plan before writing.** In Claude Code press Shift+Tab until the indicator says plan mode. Claude then shows a plan and waits for you before it changes anything. Skills also ask their questions one at a time, each with a recommendation, and wait for your yes before creating files.
4. **Do not redo finished work.** If a skill's document already exists and looks current, help will not recommend that skill again.

## Guardrails

Nothing may be invented. These rules come from the repository's `AGENTS.md` and the skills themselves.

- **Unknowns before invention.** If a decision needs real user input, the answer goes on a line in `UNKNOWNS.md`, not into a guess. Answers move to the document that owns them once confirmed.
- **Confirmed beats invented.** Use something real: a file, a spec, your research. If nothing exists, a default is allowed only when it is labelled as a guess.
- **Name the source.** Every recommendation or finding points to the file, screen or your words that produced it.
- **No made-up people, quotes, numbers or competitors.** Interview quotes need an interview number. Competitors need a real source.
- **Stay in scope.** Audit never edits. Setup adds to existing files and does not overwrite them. Renaming an existing folder needs your explicit yes.
- **Never claim what did not happen.** A skill must not say a file was written, a command run or a tool called when it was not.
- **Safe settings.** Keep Claude on its default, which asks before editing. Do not switch on bypass or auto-accept. Never paste passwords, keys or tokens into a chat.

Paste this before any skill run to enforce them:

```
Use only files in this project and the skill's own SKILL.md. Do not invent users, quotes,
numbers, competitors or screens. If something is missing, add it to UNKNOWNS.md and ask m
e.
Show me a plan first and wait for my yes before creating or changing files. Name the sourc
e
file for every finding. If you guess, say it is a guess.
```

## Avoiding slop

### In text

- Praise words with nothing behind them: seamless, powerful, vibrant, pivotal. Say what happens instead.
- Big words for plain ones: leverage, utilize, facilitate, crucial. Use "use", "help", "important".
- Vague sources: "studies show", "experts believe". Name the source or cut the claim.
- Lists forced to three items, decorative em dashes, "it's worth noting", chatbot closers such as "I hope this helps".
- If you gave the text, it stays verbatim. Claude formats it and does not rewrite it.

### In design

- No filler sections, placeholder copy or invented stats. Every element earns its place, and Claude asks before adding material.
- No aggressive gradients, emoji or stock-font defaults unless your brand uses them.
- Claude cannot generate photos. It says so and uses labelled placeholders.
- Small request, small change. If you ask for one edit, only that edit happens.

## If something goes wrong

| What you see | What to try |
|---|---|
| Typing `/` shows no skills | Quit Claude fully and reopen it. Check the installer said 30 skills. |
| `/help` opens Claude Code's own help | The built-in wins. Type: Use the AIX Lab-Atlas help skill. |
| A skill writes before showing a plan | Type: stop, show me your plan first. |
| A skill says it cannot write files or run a command | That is the Environment rule working. Run it in a place that has a file system or shell, or take the output it returns in the conversation. |
| `./install.sh` says permission denied | Run `chmod +x install.sh`, then the installer again. |